

ECOM EXPERTS

Ecom redis

Manual de usuario, tecnico y de desarrollo

ecom_redis 1.2.7 · 08/09/2026

Contenido

Configuración Inicial (Ejemplo Socket cPanel)

Configuración de Optimización TTFB

¿Cómo activarlo?

¿Qué sucede al activarlo?

Sincronización Inteligente de Limpieza

Pestañas del Módulo

Soporte

Arquitectura de Caché

Optimizaciones Avanzadas (v1.3.0+)

Detalles de Implementación

Arquitectura Híbrida (LiteSpeed + Redis)

Sincronización

Optimización PageSpeed (v1.2.4+)

Sistema de Autocuración (v1.2.4+)

Sincronización con Limpieza de PrestaShop (v1.3.1+)

Seguridad y Robustez (v1.4.0+)

1. Protección CSRF en Back Office

2. Escritura Atómica de Configuración

3. Gestión Segura de Cookies

4. Acceso Directo a Caché (getRaw)

5. Crawler de Calentamiento Seguro

6. Bypass Total por IP (Lista Blanca)

Nuevas APIs y Helpers Internos

1. EcomRedisQueryCache::getRaw(\$key)

2. checkAjaxToken()

3. Escritura Atómica con Backup

4. Logger Autocontenido

5. EcomRedisManager::isWhitelistedIp()

Flujo de Trabajo del Pre-Bootstrap

Gestión de Cookies e Invalidación

Sincronización Automática (Clean Cache)

Desarrollo de Nuevos Hooks

Optimizaciones Pro de Redis

Estándares Gravity

Configuración Inicial (Ejemplo Socket cPanel)

Para entornos de hosting compartido (como cPanel con Sockets), utilice estos valores recomendados: -

Activar Redis: Sí (Enabled). - **Host / Socket:** `/tmp/redis.sock` (o la ruta que le facilite su proveedor). -

Puerto: `0` (Imprescindible usar 0 para conexiones por Socket). - **Contraseña:** Dejar en blanco (a menos que su proveedor le asigne una). - **Base de datos:** `0`. - **Prefijo:** `ps_` (o cualquier prefijo descriptivo).

Configuración de Optimización TTFB

La nueva pestaña **Optimización TTFB** permite activar el motor de "Pre-Bootstrap". Esta técnica es la más avanzada para mejorar la velocidad de carga (Time To First Byte).

¿Cómo activarlo?

1. Vaya a la pestaña **Optimización TTFB**.
2. Active el interruptor **Activar Pre-Bootstrap Engine**.
3. Pulse **Guardar**.

¿Qué sucede al activarlo?

El módulo modificará su archivo `.htaccess` para interceptar las visitas antes de que PrestaShop empiece a cargar archivos pesados. Si la página ya está en la memoria de Redis, se servirá instantáneamente (habitualmente en menos de 50ms).

Sincronización Inteligente de Limpieza

Ya no necesita preocuparse por limpiar Redis manualmente. Cuando pulsa el botón "**Borrar caché**" de PrestaShop (en Parámetros Avanzados > Rendimiento), el módulo detecta esta acción y vacía automáticamente la caché de Redis. Esto garantiza que su sitio siempre muestre las últimas versiones de sus imágenes, CSS y textos de forma armonizada.

Pestañas del Módulo

- **Dashboard:** Vista general del consumo de memoria y ratio de aciertos (Hits).
- **Page Cache (FPC):** Configuración de tiempos de vida (TTL) para cada tipo de página y gestión de limpieza manual.
- **Rendimiento Real:** Gráficas de comparación entre tiempos de carga con caché y sin caché.
- **Configuración Redis:** Datos de conexión a su servidor Redis (Host, Puerto, Puerto, etc).
- **Caché Navegador Extrema:** Activa cabeceras `Expires` y `Cache-Control` agresivas (1 año) en el `.htaccess`. Incluye una regla para eliminar cookies en recursos estáticos, acelerando la carga.
- **Dominios sin cookies:** En la pestaña de optimización encontrará una guía para configurar Media Servers (Servidores de Medios) de PrestaShop, lo cual es altamente recomendado para el rendimiento.

- **Activar Log de Depuración:** Si experimenta errores, active esta opción. Los eventos se guardarán en `/modules/ecom_redis/logs/debug.log`.
- **PHP Opcache:** Herramientas para gestionar la memoria caché de scripts de PHP.
- **cPanel / WHM PECL:** Guía paso a paso para activar la extensión Redis en servidores con cPanel o administración WHM.
- **Exclusión de Contenido:** Permite definir rutas (URLs) o controladores específicos (como `cart`, `order`, `customer`) que NUNCA deben ser cacheados.
- **IPs Lista Blanca (Bypass Total):** Permite definir una lista de IPs que nunca verán contenido cacheado. Esta opción es fundamental para desarrolladores que necesitan ver cambios en tiempo real o realizar pruebas de rendimiento sin la interferencia de la caché.
- **Optimización TTFB:** Control del motor de pre-carga rápida para un rendimiento extremo.

Soporte

Si tiene problemas con la activación del Pre-Bootstrap, asegúrese de que su servidor tiene permisos de escritura en el archivo `.htaccess`. Puede contactarnos en ecomyseo@gmail.com.

Documentación Técnica - Ecom Redis Manager

Arquitectura de Caché

El módulo utiliza una arquitectura de tres niveles para maximizar el rendimiento de PrestaShop:

1. Nivel 0: Pre-Bootstrap Engine (TTFB < 50ms)

- Implementado en `fpc_bootstrap.php`.
- Intercepta la petición desde `.htaccess` mediante `auto_prepend_file`.
- **Validación de Assets:** Antes de servir, verifica que los archivos CCC (CSS/JS) existan físicamente.
- **Control de Versión:** Comprueba la versión global de la caché para invalidación masiva instantánea.
- No carga el núcleo de PrestaShop.
- Utiliza scripts LUA atómicos para recuperación ultra-rápida.
- Detección de sesión mediante la cookie `ecom_no_cache`.
- Soporte para patrones de exclusión de URLs configurables.
- Validación de controladores excluidos para evitar servido dinámico erróneo.

2. Nivel 1: Full Page Cache (FPC)

- Implementado en `classes/EcomRedisPageCache.php`.
- Hook: `actionDispatcherBefore` (serve) y `actionOutputHTMLBefore` (store).
- **Validación Robusta:** Escanea el HTML servido buscando recursos críticos. Si faltan en el disco, aborta el HIT y regenera la página.
- Invalidación por etiquetas (Tags) para productos, categorías y CMS.

3. Nivel 2: Query / Object Cache

- Implementado en `classes/EcomRedisQueryCache.php`.
- Cachea resultados de bases de datos y objetos serializados.

Optimizaciones Avanzadas (v1.3.0+)

- **UNLINK (Background Delete):** Para borrar grandes lotes de claves, el módulo utiliza `UNLINK` en lugar de `DEL` (siempre que el servidor Redis sea v4.0+ y la extensión lo soporte), lo que permite que el borrado ocurra en segundo plano sin bloquear el hilo principal de Redis.
- **Protección contra Cache Stampede (Gen Lock):** Implementa un bloqueo de generación ("Gen Lock") para evitar que múltiples procesos PHP regeneren la misma página simultáneamente cuando expira la caché. Los procesos concurrentes esperan unos milisegundos a que el primer proceso almacene el resultado en Redis.
- **Serialización Igbinary:** Si la extensión `igbinary` está disponible, el módulo la utiliza para reducir drásticamente el tamaño de los datos en Redis y acelerar el proceso de serialización/deserialización.

Detalles de Implementación

- **LUA Scripts:** Se utilizan para reducir los viajes de ida y vuelta (round-trips) a Redis y asegurar atomicidad en el conteo de hits/misses.
- **Detección de Dispositivo:** Diferencia entre móvil (m) y escritorio (d) en la clave de caché.
- **Invalidación Selectiva:** Al actualizar un producto, se invalidan solo las páginas relacionadas (Home, Producto, Categorías asociadas).
- **Gestión de Carritos:** El sistema monitorea hooks de carrito (`actionCartSave` , `actionCartUpdateQuantityBefore` , `actionCartDelete`) para forzar la exclusión del FPC mediante una cookie de sesión robusta.

Arquitectura Híbrida (LiteSpeed + Redis)

El módulo permite una estrategia de caché en cascada: 1. **L1: LiteSpeed (Server Level):** Responde en < 10ms para el 99% de invitados. 2. **L2: Redis ULTRA-HIT (Bootstrap Level):** Responde si L1 falla, sin cargar PrestaShop. 3. **L3: MySQL Query Cache (Redis Level):** Reduce la carga de base de datos cuando la página es dinámica (ej: usuario logueado).

Sincronización

Cuando se activa la "Sincronización con LiteSpeed Cache", el módulo envía automáticamente cebeceras `X-LiteSpeed-Purge` y ejecuta el hook `actionPurgeLiteSpeedCache` cada vez que una entidad (producto, categoría o CMS) se actualiza. Esto garantiza que el contenido en el servidor LiteSpeed sea siempre idéntico al almacenado en Redis.

Optimización PageSpeed (v1.2.4+)

El módulo procesa el HTML en `hookActionOutputHTMLBefore` antes de almacenarlo en Redis. Esto asegura que las optimizaciones ya estén "horneadas" en la caché. Todas las opciones son reversibles (SÍ/NO):

- **Font Swap:** Inyecta `&display=swap` en todas las URLs de Google Fonts del HTML para eliminar el "texto invisible" (FOIT).
- **Smart Defer JS:** Añade `defer` a scripts externos y de módulos, protegiendo jQuery, el tema y scripts ya marcados como `defer` / `async` .
- **Motive.co Lazy:** Convierte el `<script>` de Motive en un cargador basado en interacción (scroll, clic, tecla). El robot de Google no lo ejecuta, mejorando la puntuación.
- **Browser Cache (1 año):** Escribe reglas `mod_expires` y `mod_headers` en el `.htaccess` con `max-age=31536000`, `immutable` . Seguro con CCC (PrestaShop usa hashes únicos en nombres de ficheros).

Sistema de Autocuración (v1.2.4+)

- **Detección en Frontend:** `perf.js` monitoriza errores de carga en archivos del directorio `/cache/` (CCC).
- **Señal Recovery:** Si detecta un fallo, envía `recovery: true` al controlador `perf.php` .

- **Crawler Automático:** El servidor borra la caché de esa URL y ejecuta un `warmUp()` (cURL interno) para regenerarla con los assets actualizados.

Sincronización con Limpieza de PrestaShop (v1.3.1+)

- **Hook** `actionClearCache` : Detecta cuando se pulsa "Borrar caché" y vacía automáticamente Redis.

Seguridad y Robustez (v1.4.0+)

1. Protección CSRF en Back Office

Todos los endpoints AJAX del módulo (`ajaxProcess...`) están protegidos mediante una verificación estricta de token de administración. Esto evita ataques de falsificación de petición en sitios cruzados que podrían comprometer la integridad de la caché (ej: vaciado malicioso o alteración de configuración). - **Función Clave:** `checkAjaxToken()` .

2. Escritura Atómica de Configuración

Para evitar la corrupción de archivos críticos (`settings.php` , `.htaccess` , `.user.ini`) durante picos de tráfico o fallos del servidor, el módulo utiliza ahora un sistema de escritura atómica: 1. Se genera un archivo temporal. 2. Se realiza una copia de seguridad (`.bak`) del archivo original. 3. Se utiliza `rename()` (operación atómica en el sistema de archivos) para reemplazar el original.

3. Gestión Segura de Cookies

La cookie de control de caché `ecom_no_cache` se gestiona ahora con flags de seguridad modernos: -

HttpOnly: Evita el acceso desde JavaScript (protección contra XSS). - **SameSite=Lax:** Mejora la privacidad y seguridad en navegaciones desde otros sitios. - **Secure:** Se activa automáticamente si el sitio utiliza HTTPS.

4. Acceso Directo a Caché (getRaw)

Se ha implementado `EcomRedisQueryCache::getRaw($key)` para permitir que los Overrides de PrestaShop (como `Configuration`) accedan a Redis con la mínima latencia posible, gestionando internamente la descompresión y deserialización segura de datos.

5. Crawler de Calentamiento Seguro

El método `warmUp()` mantiene ahora la verificación SSL activa (`CURLOPT_SSL_VERIFYPEER=true`) por defecto, garantizando que el crawler no sea vulnerable a ataques MITM (Man-in-the-Middle) incluso en entornos locales mal configurados.

6. Bypass Total por IP (Lista Blanca)

Se ha implementado un sistema de "Lista Blanca" que permite definir IPs (ej: desarrolladores, administradores) que omiten completamente TODA la infraestructura de caché del módulo. - **Alcance:** Afecta a Nivel 0 (Bootstrap), Nivel 1 (FPC), Nivel 2 (Query Cache) y al override de `Configuration` . -

Funcionamiento: Cuando se detecta una IP autorizada, el módulo desactiva dinámicamente sus

interceptores, forzando a PrestaShop a cargar datos frescos de MySQL y ejecutar PHP sin intervención de Redis. - **Utilidad:** Permite realizar profiling preciso de lentitud en base de datos, depurar errores de lógica que podrían estar enmascarados por la caché y activar el profiling nativo de PrestaShop sin interferencias.

Guía Profesional de Desarrollo - Ecom Redis Manager (v1.4.0)

Nuevas APIs y Helpers Internos

1. EcomRedisQueryCache::getRaw(\$key)

Permite acceso de bajísimo nivel a Redis. Ideal para Overrides o procesos que se ejecutan antes del bootstrap completo de PrestaShop. Gestiona internamente la descompresión `gzq:` y la deserialización segura.

2. checkAjaxToken()

Método en la clase principal para validación centralizada de CSRF en controladores y handlers AJAX. Garantiza que solo administradores autenticados con tokens válidos ejecuten acciones críticas.

3. Escritura Atómica con Backup

Implementación de `writeAtomicWithBackup($path, $content)` para manipulación segura de archivos de configuración. Reduce a cero el riesgo de archivos corruptos por fallos de escritura concurrentes.

4. Logger Autocontenido

`EcomRedisLogger` ahora gestiona automáticamente el directorio `/logs` del módulo, crea archivos `index.php` de seguridad y realiza rotaciones automáticas al superar los 10MB.

5. EcomRedisManager::isWhitelistedIp()

Centraliza la lógica de bypass total. Este método debe ser consultado en cualquier punto de entrada que intercepte el flujo de ejecución (Dispatcher, Overrides) para garantizar que la "Lista Blanca" se respeta en todas las capas.

Flujo de Trabajo del Pre-Bootstrap

El archivo `fpc_bootstrap.php` es el punto de entrada para la optimización extrema del TTFB. Su flujo es: 1. **Validación de elegibilidad:** Solo peticiones GET, no AJAX, no estáticos. 2. **Configuración ligera:** Lee `config/settings.php` (generado por el módulo). 3. **Conexión Redis:** Usa persistencia si está configurada. 4. **Recuperación LUA:** Ejecuta script LUA para obtener el HTML y actualizar estadísticas en un solo paso. 5. **Output:** Si hay HIT, inyecta `window.ecom_ultra = true` y finaliza la ejecución con `exit`.

Gestión de Cookies e Invalidación

Para que el motor de pre-bootstrap sepa cuándo evitar la caché sin cargar PrestaShop, el módulo gestiona la cookie `ecom_no_cache`: - Se crea con `value=1` cuando hay productos en el carrito (`actionCartSave` ,

`actionCartUpdateQuantityBefore`). - Se crea al hacer login (`actionCustomerLoginAfter`). - Se elimina al vaciar el carrito (`actionCartDelete`) o hacer logout. - Utiliza parámetros de seguridad (Lax, Secure, HttpOnly) para compatibilidad con navegadores modernos.

Sincronización Automática (Clean Cache)

El sistema utiliza el hook `actionClearCache` para detectar limpiezas de caché de PrestaShop (Smarty, XML, Media). Al activarse, el módulo ejecuta un vaciado total de Redis para prevenir rutas de activos rotos (CSS/JS) en el HTML cacheado.

Desarrollo de Nuevos Hooks

Si se añade un nuevo hook que modifique contenido cacheado, debe llamarse a `$this->invalidateRedisEntity($type, $id)` dentro del hook para mantener la consistencia de la caché.

Optimizaciones Pro de Redis

- **UNLINK:** En hooks de borrado masivo (`flushAll`), el sistema utiliza `UNLINK` si está disponible para evitar micro-cortes en el servicio durante limpiezas de caché grandes.
- **Igbinary:** Se utiliza para mejorar la velocidad de persistencia de datos complejos (Query Cache) reduciendo el tiempo de CPU en PrestaShop.
- **Stampede Lock:** Al desarrollar páginas de alto tráfico que consumen muchos recursos de DB, este mecanismo protege al servidor de picos repentinos de carga cuando expira la caché.

Estándares Gravity

- **Headers:** Todos los archivos PHP deben incluir la cabecera de licencia y autoría de Ecom Experts.
- **Seguridad:** Comprobación constante de `_PS_VERSION_` .
- **Compatibilidad:** Código compatible con versiones 8 y 9 de PrestaShop.